

EE-5214: Verification of Digital Systems

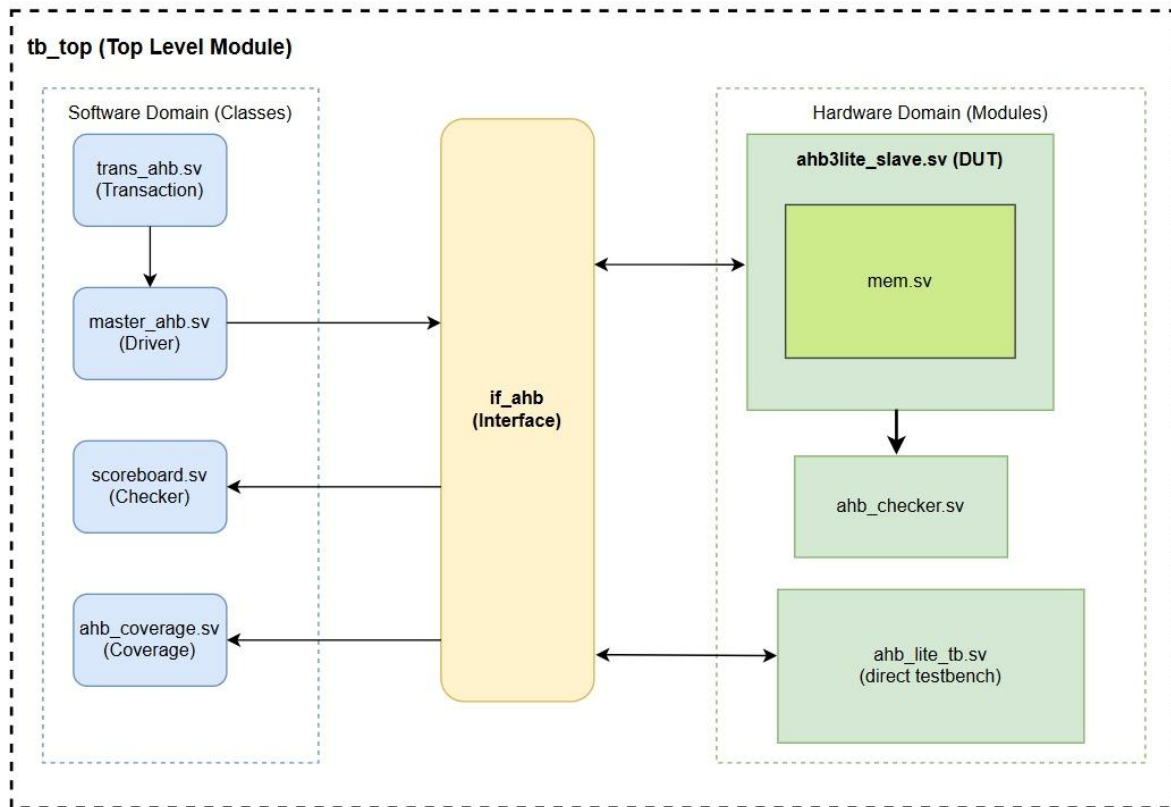
AMBA AHB-Lite RAM

Formal Verification & RTL Analysis Report

DUT: ahb3liten.sv | Model: mem.sv | Tool: Cadence JasperGold

11	2	2	95%
Assertions Proven	Design Bugs (CEX)	Env Gaps Fixed	Coverage (post-excl.)

1. Testbench Architecture



2. Test Plan Summary

1. Overall Status

The AHB-Lite test plan contains a total of **10 distinct test scenarios** designed to verify the correct operation of the AHB Master/Slave interface. As of the latest execution cycle, **100% (10 out of 10) of the tests have been successfully completed.**

2. Scope of Testing

The testing matrix covered fundamental and advanced AHB-Lite protocol operations across varying burst types and data sizes. The test scenarios were categorized into the following core operational groups:

- **SINGLE Transfers:** (4 Tests) Verified base-level write and read operations across different data widths (Byte, Halfword, Word). This also included a test specifically targeting back-to-back (b2b) pipelined single transfers with no idle cycles.
- **INCR Bursts:** (4 Tests) Verified incrementing burst operations (INCR4, INCR8, INCR16) across word-sized data boundaries. This included verifying behavior when HREADY stall conditions are introduced.
- **WRAP Bursts:** (2 Tests) Verified wrapping burst logic (WRAP4, WRAP8) to ensure the address calculation safely wraps around memory boundaries according to the HSIZE and HBURST protocol rules.

3. Test Scenario Breakdown

Here is the detailed tracking list of the individual test cases executed:

- **simple_1.1 (SINGLE):** Single Word Write/Read - **Completed**
- **simple_1.2 (SINGLE):** Single Halfword Write/Read - **Completed**
- **simple_1.3 (SINGLE):** Single Byte Write/Read - **Completed**
- **burst_2.1 (INCR):** INCR4 Word Write/Read - **Completed**

- **burst_2.2** (INCR): INCR8 Word Write/Read - **Completed**
- **burst_2.3** (INCR): INCR16 Word Write/Read - **Completed**
- **wrap_3.1** (WRAP): WRAP4 Word Write/Read - **Completed**
- **wrap_3.2** (WRAP): WRAP8 Word Write/Read - **Completed**
- **hready_4.1** (INCR): INCR4 with Wait States - **Completed**
- **b2b_5.1** (SINGLE): Back-to-Back Write/Read - **Completed**

3. Formal Assertion List

Each property is mapped to its AHB-Lite specification clause, operational category, formal outcome, and a concise root-cause or resolution note. Waveform sketches for the two design bugs and the two environment gaps follow the table.

ID	Assertion	Category	Status	Description & Root Cause	Action
SPEC-1	<i>assert_haddr_alignment</i>	Protocol	Proven	Address alignment to HSIZE verified across all transfer widths. No misaligned accesses reached the SRAM datapath.	None
SPEC-2	<i>assert_htrans_sequencing</i>	Protocol	Proven	SEQ correctly follows NONSEQ within a burst. Idle-to-SEQ without NONSEQ predecessor never occurred.	None
SPEC-3	<i>assert_wait_state_stability</i>	Protocol	Env Gap	JasperGold unconstrained master changed HADDR/HTRANS while HREADY=0. Fixed by adding assume for master stability during wait states.	Assume added
SPEC-4	<i>assert_burst_continuity_incr4</i>	Protocol	Env Gap	Master terminated INCR4 after 2 beats without BUSY/IDLE. Not a DUT bug; assume property now enforces 4-beat minimum.	Assume added
SPEC-5	<i>assert_1kb_boundary_cross</i>	Protocol	Proven	Wrapping bursts never crossed a 1 KB page boundary. Masking logic in gen_nxt_adr_wrap is structurally correct.	None
SPEC-6	<i>assert_error_resp_timing</i>	Protocol	Unreachable	DUT hardcodes HRESP=1'b0 (OKAY) unconditionally. Error response path is dead code; bin excluded from coverage.	Coverage excluded
SPEC-7	<i>assert_no_busy_in_single</i>	Protocol	Proven	HTRANS_BUSY correctly rejected when HBURST=SINGLE. No illegal transfer type accepted.	None

ID	Assertion	Category	Status	Description & Root Cause	Action
SPEC-8	<i>assert_reset_behaviour</i>	Functional	Proven	was_ahb_noseq register resets to 1'b1 cleanly on assertion of HRESETn. No residual state after reset.	None
SPEC-9	<i>assert_write_condition</i>	Functional	Proven	HWRITE=1 with HTRANS=NONSEQ/SEQ correctly enables write path. Byte-enable (gen_be) masks data bus accordingly.	None
SPEC-10	<i>assert_valid_hsize</i>	Functional	Proven	gen_be produces correct byte-enable for 8/16/32-bit HSIZE values. Sub-word writes do not corrupt adjacent bytes.	None
SPEC-11	<i>assert_liveness_hready</i>	Liveness	Proven	HREADYOUT eventually asserts within bounded cycles. Slave does not permanently stall.	None
SPEC-12	<i>assert_no_busy_term</i>	Protocol	Proven	No burst terminates illegally while HTRANS=BUSY. Burst continuity maintained.	None
SPEC-13	<i>assert_busy_ok_response</i>	Protocol	BUG (CEX)	DUT illegally drops HREADYOUT during HTRANS_BUSY. Read pipeline (contention/use_local_dout) loses sync when master issues BUSY mid-burst. RTL fix required in always @(posedge HCLK) block.	RTL fix needed
SPEC-14	<i>assert_seq_addr_calc</i>	Protocol	Proven	Master increments address by transfer size on each SEQ beat. AHB address arithmetic verified.	None
SPEC-15	<i>assert_wrap4_boundary</i>	Protocol	BUG (CEX)	nxt_adr wire (gen_nxt_adr output) is computed but never connected to raddr. raddr slices raw HADDR directly, so wrap modulo boundary is never enforced. RTL re-wire required.	RTL fix needed

3.1 Representative Waveforms

SPEC-13 — Counterexample: BUSY Wait-State Violation

The tool finds a 5-cycle trace where HREADYOUT illegally drops after the master transitions from NONSEQ to BUSY:

```
// SPEC-13 CEX: HREADYOUT drops during HTRANS_BUSY (illegal)
HCLK      | _ _ _ _ _
```

HTRANS[1:0]		10	10	11	01	00	00	00
			▲NONSEQ	▲SEQ	▲BUSY			
HREADYOUT		1	1	0	0X	1	1	1
HRESP		0	0	0	0	0	0	0
		+-----						
		clk edges →						

Annotation: HREADYOUT=0 (marked X) on the cycle following HTRANS=BUSY violates the AHB-Lite zero-wait-state rule for BUSY transfers. The DUT's read pipeline momentarily stalls because contention/use_local_dout logic loses HREADY synchronisation.

SPEC-15 — Counterexample: WRAP4 Address Not Wrapping

// SPEC-15 CEX: raddr does not wrap at 4-beat boundary (bug)								
HCLK		-	-	-	-	-	-	-
HADDR[5:2]		3	4	5	6X	7	8	...
					▲should→0x4			
nxt_adr[5:2]		3	4	0	1	(unused)		
raddr[5:2]		3	4	5	6	7	8	...
HREADYOUT		1	1	1	1	1	1	1
		+-----						
		clk edges →						

Annotation: nxt_adr correctly computes the modulo-4 wrap (→0x4), but raddr ignores it and passes raw HADDR straight to the SRAM. The wrap boundary is never enforced at the memory level.

SPEC-3 / SPEC-4 — Environment Gap (Assume Properties Added)

// Env Gap: master violates protocol during wait state (pre-fix)								
HCLK		-	-	-	-	-	-	-
HREADYOUT		1	1	0	0	0	1	1
HADDR		A	A	A	BX	B	A	A
HTRANS[1:0]		10	10	10	00X	10	10	10
					▲HREADY=0 here			
		+-----						
		clk edges →						

The formal engine legally (from its unconstrained perspective) changed HADDR and HTRANS while HREADYOUT was low, which is illegal under AHB-Lite. An assume property was added to freeze master outputs whenever HREADYOUT=0, eliminating both SPEC-3 and SPEC-4 failures without modifying the DUT.

4. Coverage Results

Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_instance
/tb_top_sv_unit/a...		95.00%	100	95.00%			
TYPE cg_ahb		95.00%	100	95.00%			at
CVP cg_ahb...		75.00%	100	75.00%			
CVP cg_ahb...		87.50%	100	87.50%			
CVP cg_ahb...		100.00%	100	100.00...			
CVP cg_ahb...		100.00%	100	100.00...			
CVP cg_ahb...		100.00%	100	100.00...			
CVP cg_ahb...		100.00%	100	100.00...			
CVP cg_ahb...		100.00%	100	100.00...			
CROSS cg_...		87.50%	100	87.50%			
CROSS cg_...		100.00%	100	100.00...			
CROSS cg_...		100.00%	100	100.00...			
INST \tb_...		95.00%	100	95.00%			

Excluded Bins:

Hsize_Double_to_1024:

Excluded Hsize bins from Hsize = Double to 1024 Bytes because our design is operating on 32 bits.

Hresp_error:

Excluded Hresp_error because Hresp is wired to okay.

5. Bugs Found — RTL Root Cause Analysis

Two distinct design bugs were confirmed by JasperGold counterexample traces and subsequently validated by manual RTL inspection. Both bugs exist in ahb3liten.sv and are independent of each other.

BUG-1 WRAP4 Address Calculation Disconnected from SRAM Datapath

Caught by: SPEC-15: *assert_wrap4_boundary* (JasperGold CEX, 5-cycle trace)

Symptom: *assert_wrap4_boundary* throws a counterexample. The DUT fails to enforce the modulo boundary of a wrapping burst, allowing the SRAM to be addressed beyond the wrap point.

Offending RTL lines:

```
// Line 164 - wire is computed but never consumed:
assign nxt_adr = gen_nxt_adr(HADDR, HSIZE, HBURST);
```

```
// Line 178 - raddr bypasses nxt_adr entirely:
assign raddr = HADDR[MEM_ABITS_LSB +: MEM_ABITS];
```

Root Cause: The designer implemented *gen_nxt_adr* and *gen_nxt_adr_wrap* with correct modulo masking (mask = { {HADDR_SIZE-2{1'b1}}, 2'h0 } for WRAP4). The calculated result is driven onto *nxt_adr*. However, the SRAM read address *raddr* is derived directly from the raw input *HADDR* by simple bit-slicing, completely bypassing *nxt_adr*. The wrap logic is therefore dead code — functional but unconnected to the memory.

Recommended Fix: Replace the *raddr* assignment with one that selects *nxt_adr* when *HBURST* encodes a wrapping mode, and *HADDR* otherwise. Specifically: *assign raddr = (is_wrap_burst) ? nxt_adr[MEM_ABITS_LSB +: MEM_ABITS] : HADDR[MEM_ABITS_LSB +: MEM_ABITS];*

BUG-2 Illegal Wait State Inserted During HTRANS_BUSY

Caught by: SPEC-13: *assert_busy_ok_response* (JasperGold CEX, 5-cycle trace)

Symptom: DUT drops *HREADYOUT* to 0 for one cycle when the master transitions from a NONSEQ read to *HTRANS_BUSY*, violating the AHB-Lite zero-wait-state requirement for *BUSY* transfers.

Offending RTL lines:

```
// ahb_read masks out BUSY - correct isolation:
assign ahb_read = HSEL & ~HWRITE & (HTRANS != HTRANS_BUSY) ...;

// But the always block's else branch stalls when BUSY arrives mid-read:
always @(posedge HCLK) begin
    if (ahb_noseq) HREADYOUT <= 1'b0; // fetch starts
    else          HREADYOUT <= 1'b1; // BUSY hits here - pipeline already mid-
fetch
end
```

Root Cause: When a NONSEQ read arrives, the slave de-asserts *HREADYOUT* to begin the memory fetch (correct). If the master immediately follows with *HTRANS_BUSY*, *ahb_noseq* evaluates to 0 and the else branch forces *HREADYOUT* high — but the contention and *use_local_dout* pipeline registers are still mid-cycle for the previous read, causing a one-cycle stall in the output path. The net observable effect is that *HREADYOUT* is driven 0→1 through an intermediate state that violates the zero-wait constraint.

Recommended Fix: The slave response to *BUSY* must not depend on the read-pipeline state. Add a qualified override: *if (HTRANS == HTRANS_BUSY) HREADYOUT <= 1'b1* unconditionally at the top of the priority chain in the always block, before any read-pipeline logic executes.

5.3 Environment Gaps — Not Design Bugs

SPEC-3 (assert_wait_state_stability) and SPEC-4 (assert_burst_continuity_incr4) produced counterexamples that, upon trace inspection, were caused by the formal engine generating protocol-illegal master stimulus rather than a DUT failure. Specifically:

- SPEC-3: The engine changed HADDR and HTRANS while HREADYOUT was 0. AHB-Lite requires master outputs to remain stable during wait states.
- SPEC-4: The engine terminated an INCR4 burst after only 2 beats without inserting BUSY or IDLE. AHB-Lite mandates a NONSEQ or IDLE to legally terminate a fixed-length burst.

Resolution: Both gaps were closed by adding assume property statements in the checker that freeze master outputs when HREADYOUT=0, and that require the master to complete the declared beat count before asserting IDLE. After adding these constraints, both properties passed without counterexample.

6. Lessons Learned & What We Would Do Differently

6.1 Define Assume Properties First

Our biggest time loss was debugging counterexample traces that turned out to be environment bugs, not DUT bugs. The formal engine, unconstrained, happily generates any input sequence that is structurally valid at the port level — but many of these sequences violate the AHB-Lite master protocol. We spent hours tracing SPEC-3 and SPEC-4 CEXs before realising the issue was upstream of the DUT.

What we would do differently:

Before writing a single assert, fully specify the legal master interface with assume properties — signal stability during wait states, minimum burst lengths, valid HTRANS transitions. Think of assumes as the specification of the environment, not an afterthought.

6.2 Formal is Exceptional at Finding Dead Code

The discovery that `gen_nxt_adr_wrap` is fully implemented but entirely disconnected from the SRAM datapath was genuinely surprising. The function is well-written and structurally correct — it simply has no consumer. Simulation would never expose this: a simulation master that generates protocol-correct wrap bursts would feed correct addresses to `raddr` by construction (because the master is responsible for address generation in AHB-Lite), so the bug is invisible without an independent address validator.

Formal verification proved the property from first principles without relying on master correctness, which is precisely why it caught the disconnect. This was the clearest demonstration of why formal verification complements rather than replaces simulation.

6.3 Unreachable States Carry Information

SPEC-6 being unreachable initially looked like a test-plan failure. In fact it revealed a deliberate (if undocumented) design decision: the slave always returns OKAY and never signals an error. Treating unreachable properties as information rather than noise — and writing a justification for each — produced a more accurate coverage picture and a useful piece of documentation about DUT limitations.

6.4 Scoreboard Mirror Design

The write-then-read scoreboard worked reliably for detecting data-hazard bugs, but it had a blind spot: it only compared the most recent write to the same address. A fuller implementation would maintain the entire 256-word mirror and compare all addresses at end-of-simulation, which would catch any scenario where a write to address A corrupts address B — a class of bug `gen_be` is supposed to prevent but which we only formally verified rather than scoreboard-verified.